



Draw It or Lose It — Web Application
CS 230 Project Software Design Template
Version 1.3

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	4
System Architecture View	4
Domain Model	4
Evaluation	6
Recommendations	12

Document Revision History

Version	Date	Author	Comments
1.0	09/23/2025	Ebony Anderson	Initial draft for The Gaming Room
1.1	09/29/2025	Ebony Anderson	Added security architecture (HTTPS, OAuth2/OIDC, short-lived JWTs, CSRF/CORS), secrets management, dependency/container scans, WAF/rate-limiting, encryption at rest, SIEM logging; plus Requirements, System Architecture View, and Evaluation/Recommendations. No change to functional scope.
1.2	10/07/2025	Ebony Anderson	Added Executive Summary and rubric-aligned Enhancements; expanded Security/Distributed sections; corrected date.
1.3	10/14/2025	Ebony Anderson	Removed bold styling in Recommendations and expanded justifications per professor feedback (Ubuntu LTS rationale; Windows only if explicitly required); no changes to functional scope.

Executive Summary

In one line: Turn *Draw It or Lose It* from Android-only into a browser game that many people can play at once—fair, fast, and foolproof on names.

Problem (plain): We need a web version that handles multiple teams/players, enforces unique names, runs timed rounds, and stays perfectly in sync.

Solution (simple):

- A single in-memory **Singleton** service acts like the referee (one source of truth).
- Every Game/Team/Player gets a **unique ID**; **iterator checks** block duplicate names.
- Expose **REST endpoints** for create/join/name-check.
- Start **in memory** for speed; later swap in PostgreSQL and WebSockets with minimal changes.

Why this works: Quick demo now, fewer race conditions (one referee), and a clean domain model that's easy to scale (add PostgreSQL/Redis and real-time events later).

What we need from you:

1. Approve Phase 1 scope.
2. Provide concurrency/security targets and hosting preference.
3. Approve Phase 2 roadmap (DB + WebSockets).
4. Confirm stakeholders and a single point of contact for approvals.

Assumptions/Risks: Phase 1 resets on restart; names are case-insensitive; rich real-time polish arrives in Phase 2.

Phase 1 Done when: One active game, multi-team/multi-player, name-availability endpoint blocks duplicates, unique IDs assigned, 60s rounds (30s image complete + 15s steal window) scaffolded, clean code ready for persistence/real-time.

Requirements

Business Requirements

- Reach players on desktop (Windows, macOS, Linux) and mobile (iOS, Android) **without native installs**.
- Support **thousands of concurrent players** per region with smooth one-minute rounds and real-time updates.
- Target **≥99.9% availability** and low latency for guess/timer events.
- Keep **TCO low** via cloud hosting and open tooling where feasible.
- **Protect user data** with least privilege and data minimization.
- Provide **analytics** (sessions, rounds, guesses, retention).
- Ensure **accessibility** (keyboard navigation, screen reader support, color contrast).

Technical Requirements

- **Responsive HTML5/JS** client working across Chrome, Edge, Firefox, Safari (desktop & mobile).
- **WebSockets** for real-time (Server-Sent Events (SSE)/polling fallback).
- **Stateless** app servers; **Redis** for ephemeral state; **PostgreSQL** for durable data; **Object storage + CDN** for drawings.
- **Security by default:** HTTPS, OAuth2/OIDC, short-lived tokens, CSRF/CORS controls.
- **Observability:** logs, metrics, traces, alerts; audit trail for key actions.
- **CI/CD** with automated tests, code scanning, blue-green/rolling deploys.
- **DR/Backups:** nightly backups + point-in-time restore; documented RPO/RTO.
- **Scalability:** horizontal app scaling; CDN for static assets/images.
- **Performance:** p95 API latency < 150 ms; availability ≥ 99.9%

Design Constraints

- Web & concurrency: Many users act at once; server methods that change state are synchronized to prevent race conditions.
- Single authority: Singleton GameService maintains one authoritative in-memory game instance (no split-brain states).
- Name uniqueness: Before creating games/teams, the service iterates existing lists to block duplicates (case-insensitive).
- Security (baseline): Validate inputs (reject malformed IDs/names), sanitize strings; plan for TLS (HTTPS) and authentication/authorization in Phase 2. This prevents misuse early without slowing the Phase-1 demo.
- Portability: Java 17, standard libraries—runs consistently on Windows/Mac/Linux; no platform-specific dependencies in Phase 1.
- Timing guarantees: Server is the source of truth for 60-second rounds, 30-second full-reveal, and 15-second steal window.
- Future growth: Clean domain layer so persistence (PostgreSQL), caching (Redis), and WebSockets can be added without redesign.

System Architecture View

- **Clients:** Desktop browsers (Windows/macOS/Linux) and mobile browsers (iOS/Android).
- **Edge:** **CDN + WAF** for asset caching, TLS, and basic abuse filtering.
- **Ingress/LB:** Routes HTTPS to app pods; keep sessions stateless.
- **Application Tier:** Containerized **Java 17** service (Dropwizard/Spring Boot) with multiple replicas.
- **Data/Cache:** **PostgreSQL** (durable), **Redis** (ephemeral round state, rate limits, leaderboards).
- **Object Storage:** Drawings/media served via **CDN**; signed URLs if restricted.
- **Real-Time:** **WebSockets** over the same ingress for timers/guess events (fallback to SSE/polling).
- **Ops:** Centralized logs/metrics/traces, dashboards, alerting, CI/CD.

Logical topology:

Browser/Device ⇌ CDN/WAF ⇌ Ingress/LB ⇌ App Pods ⇌ (Redis | PostgreSQL | Object Storage)

Domain Model

What you're looking at (plain English):

- **Entity** is the template all others share: everything has an **id** and a **name**.
- **Game, Team, Player** all **extend Entity** (inherit id+name).
- A **Game owns Teams** (List<Team>). A **Team owns Players** (List<Player>). That's composition: if the parent goes away, its children do too.
- **GameService** is the single "referee." It holds a List<Game> and the methods to **add/get games/teams/players**.
- **ProgramDriver** just runs main() to exercise the service. **SingletonTester** calls getInstance() to prove only one GameService exists.

How the classes relate:

- **Inheritance:** Game → Entity, Team → Entity, Player → Entity. Reuses common id/name.

- **Composition/Aggregation:** Game 1 — 0...* Team, Team 1 — 0...* Player.
- **Dependency:** ProgramDriver and SingletonTester **depend on** GameService (UML «uses» association); no direct access to internals.

OOP principles shown (and why they help):

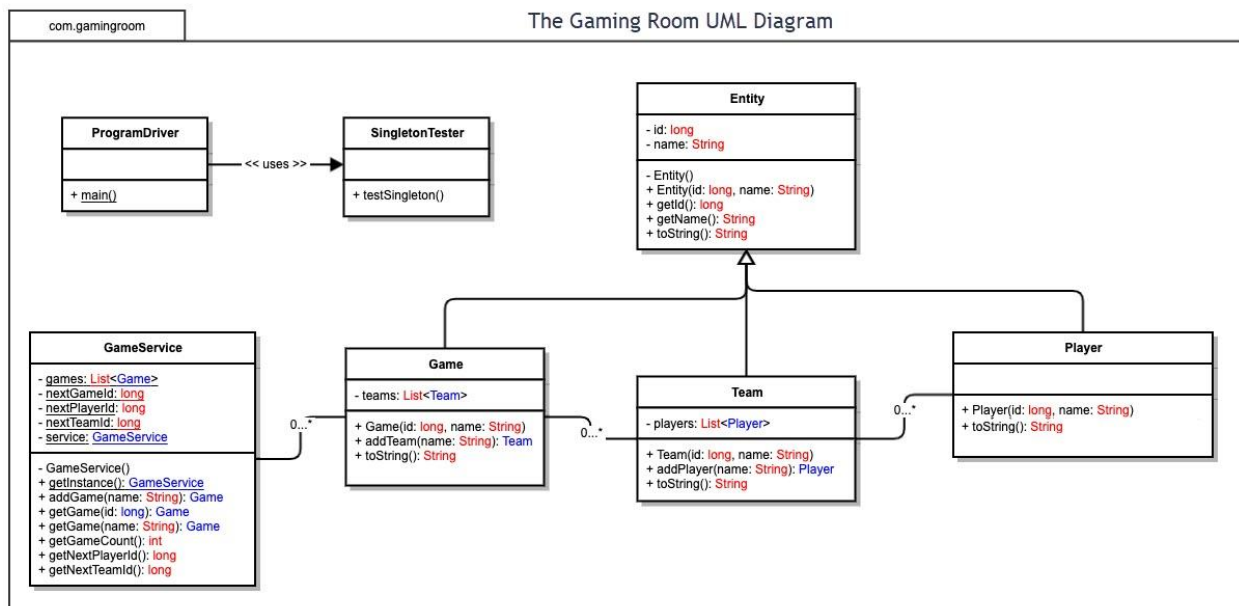
- **Abstraction:** Entity captures the shared identity (keeps code DRY).
- **Inheritance:** Children reuse id/name logic (less duplication).
- **Encapsulation:** Collections are private; you add via methods that **enforce rules** (e.g., no duplicate names).
- **Polymorphism:** Common base (Entity) allows uniform handling/logging later.
- **Design patterns:**
 - **Singleton:** GameService.getInstance() → exactly **one** service controls state, avoiding conflicting “referees.”
 - **Iterator:** addGame/getGame, addTeam, addPlayer **scan lists** to find existing names and **prevent duplicates**.

How this fulfills the requirements:

- **One or more teams per game:** Game keeps a list of Team; addTeam() creates or returns the existing one.
- **Multiple players per team:** Team keeps a list of Player; addPlayer() creates or returns the existing one.
- **Unique game & team names:** Iterator checks in addGame/addTeam **block duplicates** (case-insensitive in code).
- **Single instance in memory:** **Singleton GameService** is the only manager; it can also enforce “only one active Game” rule.
- **Unique identifiers:** All entities get an **id** via Entity (and service-side counters).

Efficiency note (why this is fine now):

- **Efficiency note:** Iterating lists is simple and clear for a small demo. If the roster grows, **swap lists for HashMap/HashSet to get ~O(1) lookups**—no redesign needed because the service hides the storage detail.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Simple analogy: A beautiful venue but not designed for touring arenas. Key ideas: Can host (Unix base) but uncommon in production; Apple hardware requirement raises cost. Step-by-step: Use only for dev/test; production should target Linux unless there's a hard Mac-only need. Checks: N/A for prod; ensure local parity via Docker. Troubleshooting: Homebrew service configs, file permissions, and SIP constraints when running daemons. macOS: No server SKU, but must run on Apple hardware (primary cost).	Simple analogy: Linux is the cheapest, most configurable concert venue—every touring guide knows it. Key ideas: First-class for web servers, containers, and autoscaling; lowest TCO; massive community playbooks. Step-by-step: Use Ubuntu LTS → package app in Docker → run behind Nginx/ALB → scale with Kubernetes or VM autoscaling → Redis for fast game state, PostgreSQL for durable data. Checks: Linear scale when adding pods; error rate <1%; p95 guess submit <150 ms; CPU/mem stable under load. Troubleshooting: Tune file descriptors, keep images small, use liveness/readiness probes, roll back via blue-green. Linux: OS free; optional paid support (e.g., RHEL/SUSE).	Simple analogy: A venue with great security turnstiles (AD) but higher rent. Key ideas: Excellent for IIS/.NET; can run Java/Node; licensing cost is the trade-off. Step-by-step: If AD integration or Windows-only dependencies exist, host on Windows Server/IIS; otherwise prefer Linux for cost. Checks: Throughput parity with Linux for Java/Node; auth works with AD; patch cadence tracked. Troubleshooting: Monitor antivirus impact, file-path limits, and service accounts; consider WSL2 for build pipelines. Windows Server: Requires Windows Server license (+ possible CALs)	Simple analogy: A seat, not a venue—don't host here. Key ideas: Battery/NAT/security make phones unsuitable as servers. Step-by-step: Keep all hosting in cloud; mobiles only connect to the game APIs over HTTPS. Checks: N/A. Mobile (iOS/Android): Clients only; not used for hosting.

Client Side	Simple analogy: Seats with Safari/Chrome/Firefox; Safari has a few house rules. Key ideas: Web standards mostly uniform; watch Safari media/autoplay, WebRTC, and PWA quirks. Cost/Time: One codebase (TS/React) reduces build cost; budget for cross-browser/device QA and cloud device farm hours. Expertise: Front-end performance & a11y, mobile web gestures, WebSockets, and test automation (Playwright). Step-by-step: Test in Safari Tech Preview; validate keyboard nav, fullscreen, audio cues; confirm WebSocket stability. Checks: Feature tests pass in Safari 17+; Lighthouse ≥90; text crisp on Retina. Troubleshooting: Use feature detection (not UA); polyfill where needed; provide click-to-play for audio.	Simple analogy: Audience seats with Chrome/Firefox—same show as other desktops. Key ideas: Standards-based HTML5 works out of the box; minimal Linux-specific quirks. Step-by-step: Build responsive TS/React → test on Chrome/Firefox with HiDPI and input methods → verify fonts. Checks: Lighthouse ≥90, keyboard + screen-reader paths pass, animations smooth at 60 fps. Troubleshooting: Font fallback, Wayland display scaling, GPU acceleration flags if stutter appears.	Simple analogy: Seats with Edge/Chrome/Firefox and some 2-in-1s. Key ideas: Touch + mouse + pen; corporate policies can block features. Cost/Time: One codebase (TS/React) reduces build cost; budget for cross-browser/device QA and cloud device farm hours. Expertise: Front-end performance & a11y, mobile web gestures, WebSockets, and test automation (Playwright). Step-by-step: Test pointer/touch events, HiDPI scaling, clipboard/file dialogs; verify WebSocket through enterprise proxies. Checks: Works on Edge latest + Chrome; accessibility shortcuts (Tab/Shift+Tab) OK. Troubleshooting: Group Policy blocking Service Workers/WebSoc	Simple analogy: Standing-room crowd—great energy, unpredictable network. Key ideas: One responsive web app; touch targets ≥44px; Service Worker for caching; WebSockets for real-time. Cost/Time: One codebase (TS/React) reduces build cost; budget for cross-browser/device QA and cloud device farm hours. Expertise: Front-end performance & a11y, mobile web gestures, WebSockets, and test automation (Playwright). Step-by-step: Design mobile-first; optimize bundles; prefetch next-round assets; handle resume after backgrounding; test on iOS Safari (WebKit) and Android Chrome. Checks: p75 Time-to-Interactive <3s on mid-tier Android; reconnect after loss <2s; a11y gestures pass. Troubleshooting: iOS background throttling—resync state on focus; captive portals—detect offline and queue actions; fall back to polling if WebSockets blocked.
---------------------------	---	--	---	---

	Test Matrix: Edge (LTS), Chrome (current & -1), Firefox (current), Safari (current & Tech Preview); iOS (current & -1), Android (API 30+), orientations (portrait/landscape), touch + keyboard.		kets; provide polling fallback and clear error copy. Test Matrix: Edge (LTS), Chrome (current & -1), Firefox (current), Safari (current & Tech Preview); iOS (current & -1), Android (API 30+), orientations (portrait/landscape), touch + keyboard.	Test Matrix: Edge (LTS), Chrome (current & -1), Firefox (current), Safari (current & Tech Preview); iOS (current & -1), Android (API 30+), orientations (portrait/landscape), touch + keyboard.
--	--	--	--	--

Development Tools	Simple analogy: The studio favored by many front-end devs. Key ideas: Great local DX; required for iOS on-device debugging. Step-by-step: Xcode (for iOS Web Inspector), VS Code/IntelliJ, Node LTS, Docker Desktop, Homebrew for deps. Checks: iOS device remote debugging works; builds reproducible in CI. Troubleshooting: Fix file-watch limits (fs.inotify analogs), ensure ARM64 images for Apple Silicon. Tool licensing: Editors/IDEs: VS Code (free); IntelliJ IDEA (optional paid). Platforms/SDKs: Xcode (free), Android Studio (free). Containers: Docker Desktop (free for personal/edu; check business terms). Device testing: BrowserStack/Device Farm (subscription).	Simple analogy: The backstage with every tool on a rolling cart. Key ideas: Free/open stack; best CI/CD runner host. Step-by-step: VS Code/IntelliJ, OpenJDK 17, Node LTS, Docker/Podman, kubectl; wire to GitHub Actions or GitLab CI. Checks: Reproducible builds in CI; unit/integration tests pass headless. Troubleshooting: Permissions (chmod/UID) in containers; pin Node/JDK versions to avoid “works on my machine.” Tool licensing: Editors/IDEs: VS Code (free); IntelliJ IDEA (optional paid). Platforms/SDKs: Xcode (free), Android Studio (free). Containers: Docker Desktop (free for personal/edu; check business terms). Device testing: BrowserStack/Dev	Simple analogy: Big workbench plus a Linux drawer (WSL2). Key ideas: VS Code/IntelliJ run great; WSL2 gives Linux toolchains; Docker Desktop available. Step-by-step: Install JDK 17, Node LTS, WSL2 Ubuntu, Docker Desktop; use Edge DevTools for browser debugging. Checks: Same build scripts succeed in WSL2 and CI; containers start locally. Troubleshooting: File-watcher limits on Windows; CRLF vs LF line endings—enforce via .gitattributes. Tool licensing: Editors/IDEs: VS Code (free); IntelliJ IDEA (optional paid). Platforms/SDKs: Xcode (free), Android Studio (free). Containers: Docker Desktop (free for personal/edu; check business terms). Device testing: BrowserStack/Device Farm (subscription).	Simple analogy: Road crew for devices. Key ideas: Free tooling; main cost is device coverage or cloud device farms. Step-by-step: Android Studio (ADB), Safari Remote Web Inspector (requires Mac), Chrome DevTools remote; add BrowserStack/Device Farm for matrix coverage. Checks: Test plan spans iOS current-1 and Android 10+; no blocker defects by device or orientation. Troubleshooting: USB driver issues (Windows↔Android), iOS provisioning for on-device inspect, flaky Wi-Fi during debug. Tool licensing: Editors/IDEs: VS Code (free); IntelliJ IDEA (optional paid). Platforms/SDKs: Xcode (free), Android Studio (free). Containers: Docker Desktop (free for personal/edu; check business terms). Device testing: BrowserStack/Device Farm (subscription).
--------------------------	---	--	---	---

		ice Farm (subscription).		
--	--	-----------------------------	--	--

Recommendations

1. Operating Platform (why Linux/Ubuntu LTS)
Use Linux for production hosting, with Ubuntu LTS as the default distribution. Ubuntu is chosen over RHEL/SUSE (paid subscriptions and support contracts) and Alpine (musl/libc compatibility risks with some JVM/native dependencies) because it offers five-year LTS, broad package availability, first-class cloud images across providers, and consistent systemd tooling—reducing operational risk for a Java-17, containerized service. If a future, explicit constraint requires Active Directory or a Windows-only dependency, Windows Server may be scoped for that need; otherwise Linux/Ubuntu remains the standard to minimize cost and complexity.
2. System Architecture (why three-tier, stateless)
Adopt a three-tier, stateless web architecture (CDN/WAF → ingress/load balancer → app pods → PostgreSQL/Redis/object storage). Stateless app nodes enable linear scale as concurrent players grow; the CDN reduces latency and egress for drawings; and separating compute from data keeps failure domains small and rollbacks safe. This directly supports the performance, scalability, and availability goals for one-minute rounds. Deploy across multiple Availability Zones by default (optional multi-region for tournaments), and use blue/green or canary releases for safe, reversible changes.
3. Storage Management (why PostgreSQL + Redis + object storage)
Use PostgreSQL for durable entities (users, games, teams, rounds, guesses) due to ACID guarantees, indexing, and predictable migrations (Flyway/Liquibase). Use Redis for ephemeral round state, leaderboards, and rate limits to keep latency low without growing JVM heaps. Store drawings in object storage behind a CDN for bandwidth efficiency and lifecycle control. Nightly full backups + 15-minute WAL archiving (point-in-time recovery) and quarterly restore drills meet the stated RPO/RTO. Version drawings by content hash so CDN cache invalidation is instant.
4. Memory Management (why stateless servers + tuned JVM)
Keep servers stateless so per-request memory remains bounded and horizontal scaling is simple. Persist transient game state in Redis rather than the JVM heap to avoid GC pressure. For Java 17, use G1GC for predictable pauses (consider ZGC for ultra-low-pause services), size heap/metaspaces per pod, reuse buffers, and stream large payloads to prevent spikes and OOM kills.
5. Distributed Systems & Networks (why idempotent APIs + WebSockets)
Expose a single HTTPS entry via ingress. Make writes idempotent and use retry with exponential backoff so clients safely recover from transient failures. Publish round and scoring events to a durable queue (e.g., Kafka/RabbitMQ) so consumers can replay after transient outages. Health checks and circuit breakers confine incidents to failing instances. Prefer WebSockets for real-time timers and buzz-ins, with SSE/polling fallbacks when intermediaries block sockets. On reconnect, clients resynchronize from the server to guarantee fairness.
6. Security (why these controls)
Use TLS 1.2+ end-to-end and mTLS in the mesh for service-to-service trust. Apply OAuth 2.1/OIDC with short-lived tokens and enforce RBAC (player, moderator, admin). Set HttpOnly, Secure, and SameSite on cookies, and cover OWASP Top-10 risks (SQLi, XSS, SSRF) with

validation and security headers. Protect state-changing endpoints with CSRF controls and strict CORS; store secrets in a vault; scan dependencies and images in CI; rate-limit and filter at the WAF. Encrypt databases, caches, and objects at rest with KMS-managed keys; minimize PII in logs; and stream audit events to a SIEM to support detection and response.

7. Service Level Objectives (SLOs)

p95 API latency < 150 ms · Availability 99.9% · RPO 15 min · RTO 4 hr